certora

Security Assessment & Formal Verification Final Report



ENSv2

May 2026

Prepared for ENS

Table of Contents

Project Summary **5**
Project Scope 5
Project Overview 5
Protocol Overview 7
Assessment Methodology **10**
Threat and Security Overview **11**
R1: Name Ownership & Token Integrity 11
R2: Access Control & Role Integrity 11
R3: Name Lifecycle & State Machine Integrity 12
R4: Registration Front-Running & Commitment Integrity 12
R5: Pricing, Payment & Economic Integrity 13
R6: Migration Safety (ENSv1 → ENSv2) 14
R7: Resolution Integrity & Traversal Safety 14
R8: HCA & Identity Resolution 15
R9: Resolver Record Integrity & Permission Granularity 15
R10: Denial of Service & Operational Liveness 16
Findings Summary 18
Severity Matrix 18
Detailed Findings **19**
Medium Severity Issues 20
M-01 Parent can overwrite emancipated child after v2 migration 20
M-02 WrapperRegistry grants ROLE_UPGRADE to frozen names 22
M-03: Expired names can still be transferred by their owners 24
Low Severity Issues 26
L-01 UUPS upgradeable contracts lack storage gaps 26
Informational Issues 27
I-01: Low level transfer instead of call 27
I-02: MIN_COMMITMENT_AGE of zero defeats front-running protection 27
I-03: Behaviour of safeTransferFrom and safeBatchTransferFrom diverge 28
I-04: Name price can be zero 28
Formal Verification **31**
Verification Methodology 31
Verification Notations 32
General Assumptions and Simplifications 32
Formal Verification Properties Overview 33
Detailed Properties 43
Commitment and Registration 43
COM_P1. Minimum commitment age 43
COM_P2. Maximum commitment age 43

COM_P3. register() reverts if duration < MIN_REGISTER_DURATION....44
COM_P4. register() reverts for invalid owner....44
COM_P5. register() reverts if name is not available....45
COM_P6. Unregistered names cannot be renewed....45
COM_P7. Non-zero rent and correct payment....45
Denial of Service....47
DOS_P1. Role assignment reverts once role count is 15....47
DOS_P2. Unregistered names cannot be renewed....47
Enhanced Access Control....49
EAC_P1. Revoking roles not currently held has no effect....49
EAC_P2. Role counts are correctly updated....49
EAC_P3. Role counts are between 0 and 15....50
EAC_P4. Grant reverts if not settable....51
EAC_P5. Role transfer is complete....52
EAC_P6. grantRoles reverts if trying to grant root roles....52
EAC_P7. Registered token’s settable roles do not include admin roles....53
Registered token’s settable roles do not include admin roles.
Assumptions: resource != ROOT_RESOURCE, getStatus(anyId) == REGISTERED....53
EAC_P8. Granting 0 roles returns false and does not modify state....53
_grantRoles with roleBitmap == 0 returns false and does not modify state.
Assumptions: resource != 0, target != 0....54
ERC1155Singleton....55
ERC_P1. Balance matches owner....55
ERC_P2. Singleton tokens are non-divisible....55
ERC_P3. Current, non-expired ids can only be transferred from their ownerOf....56
ERC_P4. Zero-valued transfer does not change ownership....56
ERC_P5. Transfer requires either from to be msgSender or msgSender be approved by from....57
ERC_P6. Owners of expired names cannot transfer the name....57
ERC_P7. Single and batch operation equivalence....58
Hidden Contract Accounts....59
HCA_P1. HCA_FACTORY configuration correctness....59
HCA_P2. HCA_FACTORY not configured correctness....59
Migration....61
MIG_P1. Locked migration reverts if name is not locked....61
MIG_P2. Locked migration reverts if parent node does not match the resolver....61
MIG_P3. Migration reverts if owner is zero....62
MIG_P4. Migration reverts if non-zero token approval....62
MIG_P5. Migration requires reserved and inherits v2 expiry....63
MIG_P6. Fuse translation covers all capability-affecting fuse bits....64
MIG_P7. Locked names cannot be migrated through the unlocked path....65
MIG_P8. Emancipated unmigrated children cannot be registered on v2....65

MIG_P9. Only v1 BaseRegistrar may migrate ERC721 tokens.... 66
MIG_P10. Only authorized operators can create reservations.... 66
MIG_P11. Reserved longer expiry renews instead of registering.... 67
Payment and Pricing.... 68
PAY_P1. rentPrice reverts for unsupported tokens.... 68
PAY_P2. rentPrice reverts for zero base units.... 68
PAY_P3. Prior owners do not pay premium.... 69
PAY_P4. Premium window is finite.... 69
PAY_P5. Discount is monotonic.... 70
PAY_P6. Invalid labels have no price.... 70
PAY_P7. Update rejects zero numerator.... 70
Registry and Ownership.... 72
REG_P1. Each ERC1155 token ID has exactly one owner.... 72
REG_P2. Transfer completeness.... 72
REG_P3. Enhanced access control and token version ids are monotonic.... 73
REG_P4. Roles move only on non-zero transfers.... 74
REG_P5. Token version bump burns and mints to the same owner.... 74
REG_P6. Granting and revoking roles increases tokenVersionId by exactly one.... 75
REG_P7. Register reverts if expiry is in the past.... 75
REG_P8. Name is available if and only if expired.... 75
REG_P9. Token operations revert when entry is expired.... 76
REG_P10. Names become immediately available after being unregistrered.... 77
REG_P11. At successful register, status and owner are updated.... 77
REG_P12. register with null-owner reserves.... 77
Resolution.... 79
RES_P1. Subregistry is zero for expired labels.... 79
RES_P2. Resolver is zero for expired labels.... 79
RES_P3. Resolver resorts to v1 names for migratable children.... 79
RES_P4. setParent cannot be called without privileges.... 80
Resolver.... 81
RSLV_P1. Clearing records resets profile getters.... 81
RSLV_P2. onlyPartRoles is a 4-step fallback over EAC resources.... 81
RSLV_P3. No record setter bypasses the permission check.... 82
Disclaimer.... 83
About Certora.... 83

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
ENSv2 (Manual Review)	contracts-v2	41b67f10(initial) 69bde1b(final)	EVM
ENSv2 (Formal Verification)	contracts-v2	e8696c67 (initial) 3cbb44c0 (final)	EVM

Project Overview

This document describes the security review and formal verification of ENS V2. The work was undertaken from **March 16th, 2026**, to **May 8th, 2026**, and comprised two complementary workstreams:

Manual Review (**March 16 – April 20, 2026**): The Certora team performed a manual security audit of all files in scope, identifying bugs and security concerns as listed in the following pages.

Formal Verification (**March 16 – May 8, 2026**): Using the Certora Prover, the team wrote formal specifications and verified that the Solidity implementation is correct with respect to those rules. During the verification process, additional bugs were discovered and are also listed below.

The following contracts were included in scope.

```
contracts/src/CommonErrors.sol
contracts/src/access-control/EnhancedAccessControl.sol
contracts/src/access-control/interfaces/IEnhancedAccessControl.sol
contracts/src/access-control/libraries/EACBaseRolesLib.sol
contracts/src/dns/DNSAliasResolver.sol
contracts/src/dns/DNSTLDResolver.sol
```

contracts/src/dns/DNSTXTResolver.sol
contracts/src/dns/libraries/DNSTXTParserLib.sol
contracts/src/erc1155/ERC1155Singleton.sol
contracts/src/erc1155/interfaces/IERC1155Singleton.sol
contracts/src/hca/HCAContext.sol
contracts/src/hca/HCAContextUpgradeable.sol
contracts/src/hca/HCAEquivalence.sol
contracts/src/hca/interfaces/IHCAFactoryBasic.sol
contracts/src/migration/AbstractWrapperReceiver.sol
contracts/src/migration/LockedMigrationController.sol
contracts/src/migration/LockedWrapperReceiver.sol
contracts/src/migration/UnlockedMigrationController.sol
contracts/src/migration/libraries/LibMigration.sol
contracts/src/registrar/BatchRegistrar.sol
contracts/src/registrar/ETHRegistrar.sol
contracts/src/registrar/StandardRentPriceOracle.sol
contracts/src/registrar/interfaces/IETHRegistrar.sol
contracts/src/registrar/interfaces/IRentPriceOracle.sol
contracts/src/registrar/libraries/LibHalving.sol
contracts/src/registry/ApprovedUpgradeGate.sol
contracts/src/registry/BaseUriRegistryMetadata.sol
contracts/src/registry/MetadataMixin.sol
contracts/src/registry/PermissionedRegistry.sol
contracts/src/registry/SimpleRegistryMetadata.sol
contracts/src/registry/UserRegistry.sol
contracts/src/registry/WrapperRegistry.sol
contracts/src/registry/interfaces/IPermissionedRegistry.sol
contracts/src/registry/interfaces/IRegistry.sol
contracts/src/registry/interfaces/IRegistryEvents.sol
contracts/src/registry/interfaces/IRegistryMetadata.sol
contracts/src/registry/interfaces/IStandardRegistry.sol
contracts/src/registry/interfaces/IWrapperRegistry.sol
contracts/src/registry/libraries/RegistryRolesLib.sol
contracts/src/resolver/AbstractMirrorResolver.sol
contracts/src/resolver/ENSV1Resolver.sol
contracts/src/resolver/ENSV2Resolver.sol
contracts/src/resolver/PermissionedResolver.sol
contracts/src/resolver/interfaces/IPermissionedResolver.sol

contracts/src/resolver/libraries/PermissionedResolverLib.sol
contracts/src/resolver/libraries/ResolverProfileRewriterLib.sol
contracts/src/reverse-registrar/L2ReverseRegistrar.sol
contracts/src/reverse-registrar/L2ReverseRegistrarWithMigration.sol
contracts/src/reverse-registrar/StandaloneReverseRegistrar.sol
contracts/src/reverse-registrar/interfaces/IContractName.sol
contracts/src/reverse-registrar/interfaces/IL2ReverseRegistrar.sol
contracts/src/reverse-registrar/libraries/ChainIdsBuilderLib.sol
contracts/src/universalResolver/UniversalResolverV2.sol
contracts/src/universalResolver/UpgradableUniversalResolverProxy.sol
contracts/src/universalResolver/libraries/LibRegistry.sol
contracts/src/utils/LibISO8601.sol
contracts/src/utils/LibLabel.sol
contracts/src/utils/LibMem.sol
contracts/src/utils/LibString.sol
contracts/src/utils/WrappedErrorLib.sol
contracts/src/utils/interfaces/IUniversalSignatureValidator.so

The Certora Prover demonstrated that the implementation of the Solidity contracts above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following pages.

Protocol Overview

ENS v2 (Ethereum Name Service version 2) is a comprehensive re-architecture of the Ethereum Name Service. It replaces the original monolithic flat registry with a hierarchical graph of individual registry contracts, where each registered name can have its own registry managing its subnames. The system is designed to support Layer 2 scalability, user-sovereign subname management, and migration from ENS v1.

The codebase consists of 60 Solidity files organized into the following modules:

Registry Layer — The core of the system. `PermissionedRegistry` is a tokenized (ERC1155) registry with resource-scoped access control (EAC — Enhanced Access Control). Each registered name becomes an ERC1155 token, and the token holder receives configurable roles (set resolver, set subregistry, transfer). Names follow a lifecycle of AVAILABLE → RESERVED → REGISTERED, with expiry-based availability. UserRegistry extends this as a UUPS upgradeable proxy for end-user subname registries. `WrapperRegistry` represents ENSv2 second (and beyond) level domains, corresponding to `NameWrapper` names in ENSv1.

Registrar Layer — `ETHRegistrar` implements a commit-reveal registration flow for .eth names with ERC-20 payment support. `StandardRentPriceOracle` computes pricing with three components: per-second base rates indexed by label length, piecewise-linear duration discounts for longer registrations, and exponential-decay premiums for recently expired names. `BatchRegistrar` allows batch reservation of names during the pre-migration phase.

Access Control Layer — `EnhancedAccessControl` (EAC) provides a bitmap-packed role system with 32 regular roles and 32 corresponding admin roles, scoped per resource. A `ROOT_RESOURCE` (resource 0) acts as a global fallback — roles granted at the root automatically apply to all resources. Each role occupies one nybble (4 bits), enabling up to 15 assignees per role per resource.

Migration Layer — `LockedMigrationController` and `UnlockedMigrationController` handle the transition from ENS v1 to v2. Users send their v1 `NameWrapper` tokens to the migration controller, which registers the name in the v2 .eth registry. `LockedWrapperReceiver` and `AbstractWrapperReceiver` provide the ERC1155 receiver logic for accepting wrapped v1 tokens.

Resolver Layer — `PermissionedResolver` is a multi-profile resolver supporting ENSIP-1 through ENSIP-19 standards (address, text, contenthash, ABI, pubkey, etc.) with fine-grained per-name permissions. `ENSV1Resolver` and `ENSV2Resolver` provide version-specific resolution, and `AbstractMirrorResolver` enables cross-resolver aliasing.

Universal Resolver — `UniversalResolverV2` is the primary client entry point. It recursively walks the registry hierarchy from the root to locate the correct resolver for any name, handling CCIP-Read for L2-hosted names.

Supporting Libraries — `LibLabel` handles label hashing and version encoding, `LibHalving` implements fixed-point exponential decay for premium pricing, `ERC1155Singleton` enforces single-supply token semantics, and `HCA` (Hierarchical Context Abstraction) provides `msg.sender` abstraction for hierarchical call contexts.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The analysis covers the core registry (`PermissionedRegistry, WrapperRegistry`), the access control layer (`EnhancedAccessControl`), the token system (`ERC1155Singleton`), the commit-reveal registrar (`ETHRegistrar`), the pricing oracle (`StandardRentPriceOracle`), the migration controllers (`LockedMigrationController`, `UnlockedMigrationController`), the batch pre-migration tool (`BatchRegistrar`), the universal resolver (`UniversalResolverV2`), and the HCA proxy identity layer (`HCAEquivalence`).

R1: Name Ownership & Token Integrity

Affected Components: `PermissionedRegistry`, `ERC1155Singleton`, `EnhancedAccessControl`
Risk Description: Each name is a singleton ERC1155 token whose ID can change (`labelhash ^ tokenVersionId`). Tokens are burned and reminted on role changes, so bugs in lifecycle, versioning, or the two "owner" views could break ownership guarantees.
Key Concerns:

1. **Singleton invariant**. One owner per token ID; `balanceOf` matches `ownerOf`.
2. **No ID collisions.** `_constructTokenId` must never map two different labelhashes to the same ID.
3. **Versions.** Counters only go up and advance correctly on unregister/regeneration.
4. **_regenerate**. The burn–mint path must not expose a transient state that breaks ownership rules.
5. **Two views**. `ownerOf` is zero for expired or version-mismatched tokens; `latestOwnerOf` reflects raw storage regardless.

Relevant Properties: REG_P1, REG_P2, REG_P3, ERC_P1, ERC_P2, ERC_P3, ERC_P4, ERC_P5

R2: Access Control & Role Integrity

Affected Components: `EnhancedAccessControl`, `PermissionedRegistry`, `RegistryRolesLib`
Risk description: Enhanced Access Control packs 64 roles (32 regular + 32 admin) into one uint256 with nybble encoding (4 bits per role, up to 15 assignees each). Effective membership is the OR of root-scoped and resource-scoped bits, so `ROOT_RESOURCE` roles apply everywhere—large trust surface for root holders. On name transfer, role handoff must be atomic and complete so the seller cannot keep privileges and the buyer does not lose them.
Key concerns:

1. `hasRoles` matches the OR of root- and resource-scoped role bits.
2. Nybble assignee counts stay aligned with real grants/revokes (no drift).
3. <= 15 assignees per role per resource is always enforced and not bypassable.

4. Only holders of the matching admin role may grant/revoke regular roles.
5. Name transfer moves roles fully: source ends with none of those roles; destination gets all; nothing lost or stuck with the wrong party.
6. Root vs resource paths stay distinct (`grantRoles` vs `grantRootRoles`).
7. After registration, admin roles on token resources cannot be granted via `grantRoles` (`_getSettableRoles` clears admin bits).

Relevant Properties: EAC_P1, EAC_P2, EAC_P3, EAC_P4, EAC_P5, EAC_P6, EAC_P7

R3: Name Lifecycle & State Machine Integrity

Affected Components: `PermissionedRegistry`, `ETHRegistrar`, `BatchRegistrar`
Risk Description: Names follow a strict lifecycle: `AVAILABLE` → `RESERVED` → `REGISTERED`, with `AVAILABLE` again after unregister or expiry. Moving between states needs the right roles (`ROLE_REGISTRAR`, `ROLE_REGISTER_RESERVED`, `ROLE_UNREGISTER`). System safety assumes this machine is strict: no skipped states, no unauthorized transitions, and expired names always treated as `AVAILABLE` on every path.
Key Concerns:

1. `register()` cannot overwrite an active registration (non-expired name that already has an owner).
2. `RESERVED` names are not re-reserved or re-registered without `ROLE_REGISTER_RESERVED` where required.
3. `renew()` never shortens the registration; expiry only moves forward (monotone).
4. Registration cannot set an expiry already in the past.
5. `setSubregistry`, `setResolver`, `grantRoles`, `revokeRoles`, and other state-changing paths respect expiry.
6. `getStatus` is `AVAILABLE` iff `block.timestamp >= entry.expiry`; the `>=` vs `>` rule matches everywhere.
7. Unregister sets expiry to `block.timestamp` so the name is immediately available.

Relevant Properties: REG_P7, REG_P8, REG_P9, REG_P10, REG_P11, REG_P12, REG_P4, REG_P5, REG_P6

R4: Registration Front-Running & Commitment Integrity

Affected Components: ETHRegistrar
Risk Description: Commit–reveal reduces registration front-running: a commitment is stored, must age within MIN/MAX bounds, registration proceeds only if the revealed parameters match

the hash. Security depends on strict timing, single-use commitments, and the hash matching every registration field.

Key Concerns:

1. Successful registration deletes the commitment so it cannot be reused in the same validity window.
2. `MIN_COMMITMENT_AGE` and `MAX_COMMITMENT_AGE` are enforced (not too soon, not too late).
3. An unexpired commitment cannot be overwritten by a new `commit()`.
4. The commitment hash binds label, owner, secret, subregistry, resolver, duration, and referrer.
5. Minimum duration is enforced.
6. Owner cannot be `address(0)` during registration.
7. Name availability is checked before the commitment is consumed.

Relevant Properties: COM_P1, COM_P2, COM_P3, COM_P4, COM_P5, COM_P6, COM_P7

R5: Pricing, Payment & Economic Integrity

Affected Components: ETHRegistrar, StandardRentPriceOracle

Risk Description:

Price = base (by label length) + piecewise-linear duration discount + exponential-decay expiry premium; ERC20 `safeTransferFrom` to an immutable beneficiary. Valid registrations must always cost a positive amount; prior-owner premium exemption must match the rules; discount must be monotone non-decreasing and premium decay monotone non-increasing to zero at `premiumPeriod`.

Key Concerns:

1. Non-zero price: ceiling rounding so no valid registration is free.
2. Payment is taken before the name exists in the registry.
3. Renewals: base rate only (no premium); base rate stays positive for valid names.
4. Premium exemption: no premium when `owner == state.latestOwner` or `owner == address(0)`.
5. Premium decay: monotone non-increasing; zero at `premiumPeriod`.
6. Discount: monotone non-decreasing.
7. Invalid labels (empty or >255 bytes): base rate zero and registration not allowed.
8. Payment-token ratio numerators and denominators are never zero.

Relevant Properties: PAY_P1, PAY_P2, PAY_P3, PAY_P4, PAY_P5, PAY_P6, PAY_P7

R6: Migration Safety (ENSv1 → ENSv2)

Affected Components: LockedMigrationController, UnlockedMigrationController, WrapperRegistry, BatchRegistrar, AbstractWrapperReceiver

Risk Description: ENSv1 NameWrapper (ERC1155) and BaseRegistrar (ERC721) become ENSv2 registry entries; BatchRegistrar pre-reserves names. Correctness needs fuse→EAC role translation that preserves user capabilities, reservation integrity through the migration window, and strict routing between locked vs unlocked migration paths.

Key Concerns:

1. Locked migration controller accepts only locked names; unlocked controller rejects locked names.
2. Provided label must hash to the NameWrapper token ID (namehash binding).
3. Migration owner cannot be address(0).
4. Migration rejects tokens with non-zero single-token approvals (frozen-approval check).
5. _inject with expiry=0 succeeds only if the name is already RESERVED.
6. Each ENSv1 fuse bit maps to a defined EAC role; admin variants are granted when CANNOT_BURN_FUSES is not set.
7. WrapperRegistry blocks direct registration of emancipated children that belong on the migration path.
8. BatchRegistrar renews (not re-registers) when the name is RESERVED with a shorter remaining expiry.
9. Unlocked controller ERC721 receiver accepts only BaseRegistrar tokens.

Relevant Properties: **MIG_P1**, **MIG_P2**, **MIG_P3**, **MIG_P4**, **MIG_P5**, **MIG_P6**, **MIG_P7**, **MIG_P8**, **MIG_P9**, **MIG_P10**, **MIG_P11**

R7: Resolution Integrity & Traversal Safety

Affected Components: `UniversalResolverV2`, `PermissionedResolver`, `LibRegistry`, `PermissionedRegistry`

Risk Description: Resolution walks the registry hierarchy top-down via `getSubregistry()` pointers. Correctness relies on expiry isolating subtrees, optional canonicity checks for apps that need back-pointer validation, and `WrapperRegistry` providing correct ENSv1 resolver fallback during migration.

Key Concerns:

1. Expired names return `address(0)` for both `getSubregistry` and `getResolver`, isolating expired subtrees.

2. `UniversalResolverV2` resolution of an expired name reverts with `ResolverNotFound`, isolating expired subtrees.
3. Canonicity validation is available via `findCanonicalName()` for applications that need it.
4. `WrapperRegistry` returns `V1_RESOLVER` for children that have not migrated yet, preserving ENSv1 resolution during transition.
5. `setParent` is gated by `ROLE_SET_PARENT` on `ROOT_RESOURCE` so canonicity back-pointers cannot be changed without authorization.

Relevant Properties: RES_P1, RES_P2, RES_P3, RES_P4

R8: HCA & Identity Resolution

Affected Components: `HCAEquivalence`, `HCAContext`, `HCAContextUpgradeable`, `ERC1155Singleton`

Risk Description: HCA overrides `_msgSender()` to map proxy wallet addresses to their underlying owners, and all ENSv2 access control depends on this identity resolution being correct and non-mutable.

Key Concerns:

1. Identity resolution matches the intended rule: a non-zero factory lookup result overrides `msg.sender`.
2. The factory address is immutable and cannot be changed after deployment.
3. If the factory address is zero, HCA resolution is disabled and `_msgSender()` falls back to `msg.sender`.

Relevant Properties: HCA_P1, HCA_P2

R9: Resolver Record Integrity & Permission Granularity

Affected Components: `PermissionedResolver`, `PermissionedResolverLib`, `ResolverProfileRewriterLib`

Risk Description: `PermissionedResolver` is a UUPS-upgradeable resolver with per-node versioned records (`addr`, `text`, `contenthash`, `ABI`, `pubkey`, `name`, `interfaces`) and fine-grained EAC permissions scoped by a 2D resource model (name/node × record part). It also supports alias-based suffix rewriting and multicall. Correctness depends on uniform permission enforcement across all setters, proper record invalidation on version bumps, and alias resolution not creating ambiguity or unbounded loops.

Key Concerns:

1. Every setter enforces `onlyPartRoles` and checks the four EAC resources in order: `(node, part)`, `(0, part)`, `(node, 0)`, `(0, 0)`.
2. `clearRecords` bumps the version and `_record(node)` reads only the current version so older records are inaccessible after a bump.
3. `getAlias` iteratively rewrites the longest suffix; only length-1 cycles are broken, longer cycles can loop to OOG; aliasing also rewrites calldata nodes via assembly in `ResolverProfileRewriterLib.replaceNode`.
4. `grantRoles` is disabled (always reverts); permissions must be granted via `grantNameRoles` / `grantTextRoles` / `grantAddrRoles` which enforce the 2D model.
5. Part-scoped grants require admin on `(node, 0)` before granting roles on `(node, part)`, and this delegation must not enable privilege escalation.

Relevant Properties: RSLV_P1, RSLV_P2, RSLV_P3

R10: Denial of Service & Operational Liveness

Affected Components: `PermissionedRegistry`, `ETHRegistrar`, `EnhancedAccessControl`
Risk Description: Protocol liveness can be impacted by resource limits (15 assignees per role/resource from 4-bit nybble counts), callback dependencies (ERC1155 onERC1155Received reverts during `burn`/`mint` regeneration), and storage pressure (commitment slots). These are design trade-offs: limits and callbacks intentionally constrain behavior but can block operations.
Key Concerns:

1. The 15-assignee cap per role per resource is enforced and cleanly reverts on overflow/underflow.
2. ERC1155 `onERC1155Received` callback behavior is well-defined: a reverting callback blocks the initiating role-change operation.
3. Commitment slots expire after `MAX_COMMITMENT_AGE` so storage cannot be occupied permanently.

Relevant Properties: DOS_P1, DOS_P2

Overall, ENSv2 security reduces to a few non-negotiable invariants: ownership and role state must stay consistent through version bumps, regeneration, transfers, upgrades, and migration; access control must be evaluated exactly as designed (including root-vs-resource scoping and HCA `_msgSender()` identity resolution); and core workflows – commit–reveal registration, state-machine transitions, pricing/payment ordering, and resolver writes – must be atomic,

correctly authorized, and expiry-boundary correct. Where the design intentionally adds constraints or dependencies (15-assignee caps, ERC1155 receiver callbacks, alias rewriting, migration fallbacks), the system must fail closed and predictably, without intermediate states or bypasses that allow privilege retention/escalation, unauthorized record changes, or invalid registration/resolution.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	3	3	3
Low	1	1	1
Informational	4	4	3
Total	8	8	7

Severity Matrix

Impact		Likelihood: Low	Likelihood: Medium	Likelihood: High
	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium

Detailed Findings

ID	Title	Severity	Status
M-01	Parent can overwrite emancipated child after v2 migration	Medium	Fixed.
M-02	`WrapperRegistry` grants `ROLE_UPGRADE` to frozen names	Medium	Fixed.
M-03	Expired name owners can still transfer them	Medium	Fixed.
L-01	UUPS upgradeable contracts lack storage gaps	Low	Fixed.
I-01	Low level transfer instead of call	Informational	Fixed.
I-02	`MIN_COMMITMENT_AGE` of zero defeats front-running protection	Informational	Acknowledged.
I-03	Behaviour of `safeTransferFrom` and `safeBatchTransferFrom` diverge	Informational	Fixed.
I-04	Name price may be zero	Informational	Fixed.

Medium Severity Issues

M-01 Parent can overwrite emancipated child after v2 migration

Severity: **Medium**	Impact: **High**	Likelihood: **High**
Files: `LockedWrapperReceiver.sol` `AbstractWrapperReceiver.sol`	Status: Fixed.	Violated Property: MIG_P8

Description: In ENSv1, the `PARENT_CANNOT_CONTROL` fuse protects a child's name from being overwritten by its parent. Once set, the parent cannot call `setSubnodeOwner` to replace the child (`NameWrapper` reverts at line 727). This guarantee holds regardless of whether the child has also burned `CANNOT_UNWRAP`. In ENSv2, after a parent migrates via `LockedMigrationController`, the parent's `WrapperRegistry` has a guard called `_isMigratableChild()` that is supposed to block registration of labels belonging to unmigrated v1 children. However, this guard only checks `CANNOT_UNWRAP`:

```TypeScript
// LockedWrapperReceiver.sol
/// @dev Determine if `label` is emancipated but not-yet migrated.
function _isMigratableChild(string memory label) internal view returns (bool) {
    bytes32 node = NameCoder.namehash(getWrappedNode(), keccak256(bytes(label)));
    (address ownerV1, uint32 fuses, ) = NAME_WRAPPER.getData(uint256(node));
    return ownerV1 != address(this) && _isLocked(fuses);
}

// AbstractWrapperReceiver.sol
/// @dev Returns `true` if the NameWrapper token is locked.
function _isLocked(uint32 fuses) internal pure returns (bool) {
    // PARENT_CANNOT_CONTROL is required to set CANNOT_UNWRAP, so CANNOT_UNWRAP is sufficient
    // see: V1Fixture.t.sol: `test_nameWrapper_CANNOT_UNWRAP_requires_PARENT_CANNOT_CONTROL()`
    return (fuses & CANNOT_UNWRAP) != 0;
}
```

The comment's assumption is backwards. While it's true that `CANNOT_UNWRAP` implies `PARENT_CANNOT_CONTROL`, the reverse is not true, and a child can have `PARENT_CANNOT_CONTROL` without `CANNOT_UNWRAP` fuse. These emancipated (but not locked) children are protected in v1 but invisible to the v2 guard.

Exploit Scenario:

This allows the parent to call `WrapperRegistry.register("child", ...)` and take ownership of the child's label in v2. The child owner has no migration path and no way to block the registration.

Recommendations:

Check `PARENT_CANNOT_CONTROL` in addition to `CANNOT_UNWRAP`:

```javascript
function _isMigratableChild(string memory label) internal view returns (bool) {
    bytes32 node = NameCoder.namehash(getWrappedNode(), keccak256(bytes(label)));
    (address ownerV1, uint32 fuses, ) = NAME_WRAPPER.getData(uint256(node));
    return ownerV1 != address(this) && (fuses & (CANNOT_UNWRAP | PARENT_CANNOT_CONTROL)) != 0;
}
```

Customer Response:

Fixed in f52cca0.

Fix Review:

Fix confirmed.

M-02 `WrapperRegistry` grants `ROLE_UPGRADE` to frozen names

Severity: **Medium**	Impact: **High**	Likelihood: **High**
Files: `WrapperRegistry.sol`	Status: Fixed.	

`WrapperRegistry.initialize()` always OR's `ROLE_UPGRADE | ROLE_UPGRADE_ADMIN` into the admin's role bitmap regardless of whether the name's fuses indicate it is maximally frozen in ENSv1.

```
JavaScript
_grantRoles(ROOT_RESOURCE, RegistryRolesLib.ROLE_UPGRADE | RegistryRolesLib.ROLE_UPGRADE_ADMIN |
roleBitmap, admin, false);
```

The roleBitmap argument passed to `initialize()` is computed by `_subregistryRoleBitmapFromFuses()` in `LockedWrapperReceiver`, which correctly respects the `CANNOT_BURN_FUSES` flag (a frozen name does not receive admin-level subregistry roles). However, `ROLE_UPGRADE` and `ROLE_UPGRADE_ADMIN` are prepended to that bitmap unconditionally, so even a maximally frozen name's `WrapperRegistry` admin holds upgrade capability.

For a name with all fuses burned, the intended semantic is that the name's protections are permanent. In ENSv2, the `WrapperRegistry` is a UUPS proxy, and `ROLE_UPGRADE` allows the admin to replace the proxy's implementation with arbitrary code. This means subname holders under a frozen parent lose their immutability guarantees and the parent admin can upgrade the proxy to bypass restrictions like `CANNOT_CREATE_SUBDOMAIN` or `CANNOT_TRANSFER`.

Recommendations:

Conditionally include `ROLE_UPGRADE | ROLE_UPGRADE_ADMIN` only when the name is not frozen:

```JavaScript
uint256 upgradeRoles =
       _notFrozen(fuses)
       ? RegistryRolesLib.ROLE_UPGRADE | RegistryRolesLib.ROLE_UPGRADE_ADMIN
       : 0;
_grantRoles(ROOT_RESOURCE, upgradeRoles | roleBitmap, admin, false);
```

This requires passing fuses into `initialize()`, or splitting the frozen vs. non-frozen initialization path in `LockedWrapperReceiver._migrateWrapped()`.

Customer response:
Fixed in 4ee4218.

Fix Review:

Fix confirmed.

M-03: Expired names can still be transferred by their owners

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: `ERC1155Singleton.sol`	Status: Fixed.	Violated Property: **ERC_P6**

Description: The current ownership check for a name is performed manually during the `_update` flow as seen below

```
JavaScript
if (value > 0) {
    address owner = _owners[id];
    if (owner != from) {
        revert ERC1155InsufficientBalance(from, 0, value, id);
    } else if (value > 1) {
        revert ERC1155InsufficientBalance(from, 1, value, id);
    }
    _owners[id] = to;
}
```

Fetching the owner from the `_owners` mapping will still return the addresses of holders of expired names so the `owner == from` check will pass. Instead, the `PermissionedRegistry.ownerOf` should be performed which returns `address(0)` in case of an expired name.

```
JavaScript
function ownerOf(uint256 tokenId) public view virtual
    override(ERC1155Singleton, IERC1155Singleton)
    returns (address)
{
    Entry storage entry = _entry(tokenId);
    return
        tokenId != _constructTokenId(tokenId, entry) || _isExpired(entry.expiry)
```

```
        ? address(0)
        : super.ownerOf(tokenId);
}
```

Exploit Scenario:

This could allow deceiving external buyers who haven't checked the expiration timestamp, but only the ownership of the NFT.

Recommendations:

Construct the resource with `eacVersionId + 1` once a name has expired

Customer response:
Fixed in 9e8042d.

Fix Review:

Fix confirmed.

Low Severity Issues

L-01 UUPS upgradeable contracts lack storage gaps

Severity: **Low**	Impact: **Low**	Likelihood: **Low**

Files:
`WrapperRegistry.sol`
`UserRegistry.sol`
`PermissionedResolver.sol`
	Status: Fixed.	

Description:

The three UUPS upgradeable contracts (`WrapperRegistry`, `UserRegistry`, `PermissionedResolver`) inherit from custom base contracts (`ERC1155Singleton`, `EnhancedAccessControl`, `PermissionedRegistry`) that use a traditional storage layout without `__gap` reserves or ERC7201 namespaced storage.

While OZ's upgradeable contracts use ERC7201 storage internally, ENS v2's own base contracts do not. If a future upgrade adds a state variable to any base contract, it would shift the storage slots of all derived contracts, corrupting the existing proxy state.

Recommendations: Add `__gap` arrays to each base contract, or migrate to ERC7201 namespaced storage.

Customer response:
Fixed in b8c0947.

Fix Review:

Fix confirmed.

Informational Issues

I-01: Low level transfer instead of call

Description: In a previous version of the code, there was an instance where native token overpayments were returned to `msg.sender` by using `.transfer` instead of `.call`. This is a standard practice, however the bug has been fixed by removing support of native token payments.

Recommendation: None, fixed

Customer Response:

Resolved pre-audit.

Fix Review:

Fix confirmed.

I-02: `MIN_COMMITMENT_AGE` of zero defeats front-running protection

Files:
`ETHRegistrar.sol`

Description:
The ETHRegistrar constructor validates that `maxCommitmentAge > minCommitmentAge` but does not enforce a minimum value for `minCommitmentAge`. If deployed with `MIN_COMMITMENT_AGE = 0`, the commit-reveal scheme provides no front-running protection.

Exploit Scenario:

An attacker can call `commit()` and `register()` in the same transaction, since `_consumeCommitment` checks `block.timestamp >= t0 + 0`, which is immediately true.

Recommendation:
Add a minimum bound check in the constructor

Customer Response:

Acknowledged.

I-03: Behaviour of `safeTransferFrom` and `safeBatchTransferFrom` diverge

Related property: ERC_P7

Files:
`ERC1155Singleton.sol`

Description:

Calling `safeBatchTransferFrom` for `n` items should result in the same revert behavior as calling `safeTransferFrom `n` times for the same items.

The property does not hold for the revert behaviour of length-two `safeTransferFrom`. `safeBatchTransferFrom(from, to, [id, id], [1, 0], data)` does not revert in general, but `safeTransferFrom(from, to, id, 1, data); safeTransferFrom(from, to, id, 0, data);` reverts (on the latter `safeTransferFrom`). This is essentially because the permissions are checked before transfers on `safeBatchTransferFrom` and the permission to transfer is already transferred to `to` in the non-batch sequence.

Fixed in b682ca4.

Fix Review:

Fix confirmed.

I-04: Name price can be zero

Related property: COM_P7

Files:
`StandardRentPriceOracle.sol`

Description:
After `register()`, exactly base+premium is deducted from the caller's payment token account. This amount is supposed to be strictly positive. However, the system applies a configurable discount to the base price, and after discount the price may go to zero under certain conditions. The issue arises in this code snippet in `StandardRentPriceOracle.sol'` function `rentPrice`.

```
JavaScript
        IPermissionedRegistry.State memory state = REGISTRY.getState(LibLabel.id(label));
        uint64 t = state.expiry > block.timestamp ? state.expiry - uint64(block.timestamp) : 0;
        baseUnits -= Math.mulDiv(
            baseUnits,
            integratedDiscount(t + duration) - integratedDiscount(t),
            uint256(type(uint128).max) * duration
        );
        uint256 premiumUnits;
```

For example, setting the discount points according to Table 1 and querying the `rentPrice` with `duration = 4` and `state.expiry - block.timestamp = 4`, the discount will be computed as `integratedDiscount(8) - integratedDiscount(4) = 6*(2^128 - 1) - 2*(2^128-1) = 4*(2^128-1)`, which, when divided by `(2^128-1) * duration = (2^128-1) * 4` gives `1`. Therefore `baseUnits = baseUnits - baseUnits = 0`.

Table 1: Example of discount points configuration where the price may go to zero

t	1	2	5
value	$2^{128}-1$	0	$2^{128}-1$

Discount points can only be set in executions passing through the internal function `_setDiscountPoints`, but this function only checks that the `t` value is non-null.

Recommendation:
Implement safeguards for `_setDiscountPoints`.

Fixed in PR #286.

Fix Review:

Fix confirmed.

Formal Verification

Verification Methodology

We performed verification of the ENS v2 protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and ENS's implementation source code written in Solidity. More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

1. Unless otherwise stated, where relevant, the properties were verified assuming up to three loop iterations. When hashing unbounded bytes arrays, the arrays were expected to be up to length five.
2. We use a mock implementation for ERC20 tokens, `BaseRegistrarV1` and `HCAFactory`, assuming that these components work as expected.
3. We use a deterministic random function as a substitution for `getAccountOwner(address)`. When it helps the modeling, we substitute bitwise operations with their precise, linear counterparts (e.g., when clearing the least significant 32 bits of a `uint256`). We use nondeterministic summaries for ERC1155 handshake functions (`onERC1155Received, onERC1155BatchReceived`). We summarize the gas-efficient array function `unsafeMemoryAccess` with a standard array lookup, and `_asSingletonArrays` with a standard array creation. We replace the assembly implementations of `PermissionedResolverLib` with their standard solidity implementations. We assume the correctness of OpenZeppelin's `mulDiv` implementations.

Formal Verification Properties Overview

ID	Title	Impact	Status
COM_P1	Minimum commitment age	A name cannot be registered before at least `MIN_COMMITMENT_AGE` has passed since the block where the commitment was done. Catches issues related to registration front running (R4)	Verified
COM_P2	Max commitment age	A name cannot be registered at or after `MAX_COMMITMENT_AGE` has passed since the block where the commitment was done. Catches issues related to registration front running (R4)	Verified
COM_P3	Duration too short reverts	`register()` reverts with `DurationTooShort` if `duration < MIN_REGISTER_DURATION`. Catches issues related to registration front running (R4)	Verified
COM_P4	Invalid owner reverts	`register()` reverts if `owner == address(0)` Catches issues related to registration front running (R4)	Verified
COM_P5	Name not available reverts	`register()` reverts if `getStatus(labelId) != AVAILABLE` Catches issues related to commitment integrity (R4)	Verified
COM_P6	Renew available names reverts	Unregistered names cannot be renewed. Catches issues related to commitment integrity (R4)	Verified
COM_P7	Names have nonzero rent	Names have non-zero rent which is correctly charged from the user.	Verified after fix

		Catches the bug described in I-04 (R4)	
DOS_P1	Role count <= 15	Role assignment reverts if granting would push any role nybble count past 15. Catches issues related to Denial of Service (R11)	Verified
DOS_P2	Commitment slot reopens after max commitment age	In `ETHRegistrar`: Commitment slots self-expire after `MAX_COMMITMENT_AGE` - no permanent storage occupation. Catches issues related to Denial of Service (R11)	Verified
EAC_P1	Revoke nonheld returns false	`revokeRoles` with roles not currently held by account returns `false` and does not modify state Catches issues related to role integrity (R2)	Verified
EAC_P2	Role count correctness	For every role `R` in resource `X`, the nybble count in `_roleCount[X]` equals the cardinality of `{account | _roles[X][account] & R != 0}` Catches issues related to role integrity (R2)	Verified
EAC_P3	Role nybbles between `0x0` and `0xF`	No role nybble in `_roleCount[resource]` exceeds `0xF (15)` - `_grantRoles` reverts with `EACMaxAssignees` before overflow. Similarly, the `_roleCount[resource]` may never decrease from `0x0`. Catches issues related to role integrity (R2)	Verified
EAC_P4	Grant reverts if not settable	`grantRoles(resource, roleBitmap, account)` reverts if `roleBitmap & ~_getSettableRoles(resource, caller) != 0` — only admin-authorized roles can be granted. Catches issues related to role integrity (R2)	Verified
EAC_P5	Transfer roles complete	After `_transferRoles(resource, src, dst, false), _roles[resource][src] == 0` and the	Verified

		destination holds all source's prior roles. Catches issues related to role integrity (R2)	
EAC_P6	Grant roles root reverts	Trying to grant roles to `ROOT_RESOURCE` through `grantRoles` reverts	Verified
EAC_P7	Registered token settable roles strip admin bits	For non-root resources on registered tokens, `_getSettableRoles` returns `adminRoleBitmap >> 128` (admin bits stripped) — post-registration role changes cannot grant admin roles Catches issues related to role integrity (R2)	Verified
ERC_P1	Balance matches owner	`balanceOf(account, id)` returns exactly 1 if `ownerOf(id) == account`, exactly `0` otherwise - never returns any other value. Catches issues related to name ownership (R1)	Verified
ERC_P2	Safe transfer reverts if value > 1	`safeTransferFrom(from, to, id, value, data)` `reverts if` `value > 1` - singleton tokens are non-divisible. Catches issues related to name ownership (R1)	Verified
ERC_P3	Safe transfer reverts if from not owner and value positive	Current, non-expired ids can only be transferred from their `ownerOf`. Catches issues related to name ownership (R1)	Verified
ERC_P4	Zero value safe transfer preserves latest owner	Transfer with `value == 0` does not change ownership. Catches issues related to name ownership (R1)	Verified
ERC_P5	Safe transfer reverts unless self or operator for msg sender	`safeTransferFrom` requires `from == _msgSender()` or `isApprovedForAll(from, _msgSender())` - operator authorization is checked	Verified

		Catches issues related to name ownership (R1)	
ERC_P6	Safe transfer of expired names reverts	Owners of expired names cannot transfer the name. Catches the bug described in Issue M-03 (R1)	Verified after fix
ERC_P7	Batch and single operations are equal	A sequence of single operations have the same behavior as the corresponding batch operation. Catches the bug described in issue I-03 (R1)	Verified after fix
HCA_P1	`msgSender` equals `getAccountOwner` when non-zero	If `HCA_FACTORY.getAccountOwner(sender)` returns non-zero, `_msgSender()` returns that address. Catches issues related to identity resolution (R8)	Verified
HCA_P2	`msgSender` equals `msg.sender` when factory zero	If `HCA_FACTORY == address(0)`, `_msgSender() == msg.sender` always Catches issues related to identity resolution (R8)	Verified
MIG_P1	Locked migrate reverts if name not locked	`migrateWrapped` reverts if name is not locked (`CANNOT_UNWRAP` fuse not burned). Catches issues related to migration safety (R6)	Verified
MIG_P2	Finish reverts on id or data mismatch	`_migrateWrapped` reverts if `ids[i] != namehash(parentNode, keccak256(label))` Catches issues related to migration safety (R6)	Verified
MIG_P3	Finish reverts if owner is zero	Migration reverts if `md.owner == address(0)` Catches issues related to migration safety (R6)	Verified

MIG_P4	Finish reverts on nonzero token approval	Migration reverts if `NameWrapper` token has non-zero single-token approval Catches issues related to migration safety (R6)	Verified
MIG_P5	Finish requires reserved and inherits v2 expiry	`_inject` with `expiry == 0` inherits reserved expiry - successful only if name is in `RESERVED` status. Catches issues related to migration safety (R6)	verified
MIG_P6	Fuse translation correctness	Fuse translation covers all capability-affecting fuse bits Catches issues related to migration safety (R6)	Verified
MIG_P7	Unlocked migrate reverts if name locked	`UnlockedMigrationController._migrateWrapped` reverts if name is locked - locked names must use `LockedMigrationController` Catches issues related to migration safety (R6)	Verified
MIG_P8	Register reverts for emancipated non migrated child	`WrapperRegistry.register()` for emancipated children that haven't migrated yet. Catches the bug described in issue M-01 (R6)	Verified after fix
MIG_P9	`onERC721Received` reverts unless caller is v1 registrar	`UnlockedMigrationController.onERC721Received` reverts if `msg.sender != address(_REGISTRAR_V1)` - only `BaseRegistrar` tokens accepted Catches issues related to migration safety (R6)	Verified
MIG_P10	Batch register only owner	`BatchRegistrar.batchRegister` is `onlyOwner` - only authorized operator can create reservations Catches issues related to migration safety	Verified

		(R6)	
MIG_P11	Reserved longer expiry renews not register	`BatchRegistrar` renews instead of re-registering if name is `RESERVED` and new expiry > current expiry Catches issues related to migration safety (R6)	Verified
PAY_P1	Unsupported token reverts	`rentPrice(label, owner, duration, paymentToken)` reverts if `_paymentRatios[paymentToken].denom == 0` Catches issues related to Economic integrity (R5)	Verified
PAY_P2	Zero base units reverts	`rentPrice` reverts if `baseRate(label) * duration == 0` Catches issues related to Economic integrity (R5)	Verified
PAY_P3	Null or prior owner zero premium	Premium is zero when `owner == address(0)` or `owner == state.latestOwner` – prior owner exemption Catches issues related to Economic integrity (R5)	Verified
PAY_P4	Premium after window zero	`premiumPriceAfter(duration)` returns `0` when `duration >= premiumPeriod` – premium window is finite Catches issues related to Economic integrity (R5)	Verified
PAY_P5	Integrated discount monotone	`integratedDiscount(t)` is monotonically non-decreasing in `t` Catches issues related to Economic integrity (R5)	Verified
PAY_P6	Empty or long label base rate	`baseRate(label)` returns `0` for labels with `bytes(label).length == 0` or `> 255` – invalid	Verified

	zero	labels have no price. Catches issues related to Economic integrity (R5)	
PAY_P7	Update rejects zero numerator	Payment ratio integrity - `numer > 0` and `denom > 0` for all active payment tokens (enforced in constructor and updatePaymentToken) Catches issues related to Economic integrity (R5)	Verified
REG_P1	`erc1155Singleton` balance iff owner	Each `ERC1155` token ID has exactly one owner Catches issues related to name ownership (R1)	Verified
REG_P2	Transfer completeness	Transfer completeness - after `safeTransferFrom(from, to, tokenId, 1, data)`, the following hold: `ownerOf(tokenId) == to`, `_roles[resource][from] == 0` or `to == from`, `_roles[resource][to]` contains all roles previously held by from as well as the roles previously held by to. Catches issues related to name ownership (R1)	Verified
REG_P3	Entry EAC and token version ids non-decreasing	`eacVersionId` and `tokenVersionId` are monotonically non-decreasing per name. They increment on `register` and `unregister`. `tokenVersionId` increases and `eacVersionId` does not change for `grantRoles` and `revokeRoles` (through a call to `_regenerate`). Catches issues related to name ownership (R1)	Verified
REG_P4	Erc1155 zero-transfer preserves roles	zero-value transfer must not strip roles. 1-value transfer must strip roles from src and add them to dst. Catches issues related to name ownership (R1)	Verified

REG_P5	Token version bump burn mint same owner	When the token version transitions, the effects are correct. Catches issues related to name ownership (R1)	Verified
REG_P6	Entry token version increase on grant or revoke	After a successful `grantRoles` / `revokeRoles`, `tokenVersionId` for that entry increases by exactly one. Catches issues related to name ownership (R1)	Verified
REG_P7	Register reverts when label expired per registry	`register()` reverts if the passed expiry is in the past or the entry's expiry is in the past. Catches issues related to name ownership (R1)	Verified
REG_P8	`getStatus` available iff expired	`getStatus(anyId)` returns `AVAILABLE` if and only if `block.timestamp >= entry.expiry` Catches issues related to name ownership (R1)	Verified
REG_P9	Token role ops revert when entry expired	`setSubregistry`, `setResolver`, `grantRoles`, `revokeRoles` on a token resource all revert if `block.timestamp >= entry.expiry` Catches issues related to name ownership (R1)	Verified
REG_P10	Unregister sets expiry to `block.timestamp`	After `unregister(anyId), entry.expiry == block.timestamp` – name is immediately available Catches issues related to name ownership (R1)	Verified
REG_P11	Register nonzero owner sets owner and registered status	After `register(label, owner, ..., expiry)` with `owner != address(0)`, `ownerOf(tokenId) == owner` and `getStatus(labelId) == REGISTERED`. Catches issues related to name ownership (R1)	Verified

REG_P12	Register zero owner sets no owner and reserved status	After `register(label, address(0), ..., expiry)`, `ownerOf(tokenId) == address(0)` and `getStatus(labelId) == RESERVED` Catches issues related to name ownership (R1)	Verified
RES_P1	`getSubregistry` zero when expired	`getSubregistry(label)` returns `address(0)` for expired names Catches issues related to resolution integrity (R7)	Verified
RES_P2	`getResolver` zero when expired	`getResolver(label)` returns `address(0)` for expired names Catches issues related to resolution integrity (R7)	Verified
RES_P3	`getResolver` returns v1 when migratable child	`WrapperRegistry.getResolver(label)` returns `V1_RESOLVER` for migratable children - ENSv1 resolution fallback during migration Catches issues related to resolution integrity (R7)	Verified
RES_P4	`setParent` reverts without appropriate role	`setParent` requires `ROLE_SET_PARENT` on `ROOT_RESOURCE` - canonicity back-pointer is privileged Catches issues related to resolution integrity (R7)	Verified
RSLV_P1	`clearRecords` resets profile getters	After `clearRecords(node)`, all prior records for that node are inaccessible via the public getters. Catches issues related to record integrity (R10)	Verified
RSLV_P2	`onlyPartRoles` fallback resources grant	The `onlyPartRoles` modifier checks 4 resources in fallback order: `resource(node, part)`, `resource(0, part)`, `resource(node, 0)`, and `ROOT_RESOURCE`. Having a role in any of	Verified

		these satisfies the check Catches issues related to record integrity (R10)	
RSLV_P3	Record setters enforce `onlyPartRoles`	Every record setter (`setAddr`, `setText`, `setContenthash`, `setABI`, `setPubkey`, `setName`, `setInterface`) enforces the `onlyPartRoles` modifier - no setter bypasses the permission check Catches issues related to record integrity (R10)	Verified

Detailed Properties

Commitment and Registration

Module General Assumptions

HCA Factory works as expected

Module Properties

COM_P1. Minimum commitment age

Status: Verified	**A name cannot be registered before at least `MIN_COMMITMENT_AGE` has passed since the block where the commitment was done**

Rule Name	Status	Description	Link to rule report
com_p1_min_commitment_age	Verified	*If a name was committed in an earlier transaction and register is called too soon — specifically while `e2.block.timestamp < commitmentAt(e2, commitment) + MIN_COMMITMENT_AGE(e2)` — then register must revert (lastReverted is true). You cannot complete registration until at least `MIN_COMMITMENT_AGE` has passed since the commitment was recorded.*	*Initial commit* *Final commit*

COM_P2. Maximum commitment age

Status: Verified	**A name cannot be registered after `MAX_COMMITMENT_AGE` has passed since the block where the commitment was done**

Rule Name	Status	Description	Link to rule report

com_p2_max_commitment_ag	Verified	*If a commitment was stored via `commit`, and `register` is attempted at or after the end of the commitment validity window — when `e2.block.timestamp >= commitmentAt(e2, commitment) + MAX_COMMITMENT_AGE()` — then `register` must revert. You cannot register using a commitment once it is too stale.*	*Initial commit* *Final commit*

COM_P3. register() reverts if duration < MIN_REGISTER_DURATION.

Status: Verified	**Minimum duration for the entries**

Rule Name	Status	Description	Link to rule report
com_p3_duration_too_short_reverts	Verified	*If register is invoked with duration strictly below the minimum registration duration, then register must revert*	*Initial commit* *Final commit*

COM_P4. register() reverts for invalid owner.

Status: Verified	**`register()` reverts if `owner == address(0)`**

Rule Name	Status	Description	Link to rule report
com_p4_invalid_owner_reverts	Verified	*if `register` is called with `owner == address(0)`, then `register` must revert.*	*Initial commit* *Final commit*

COM_P5. register() reverts if name is not available.

Status: Verified	`register()` reverts if the name is not available

Rule Name	Status	Description	Link to rule report
com_p5_name_not_available_reverts	Verified	*If the name is not available at registration time — formally `!isAvailable(e, label)` — then register must revert. You cannot register when the registry reports the label as unavailable.*	Initial commit Final commit

COM_P6. Unregistered names cannot be renewed

Status: Verified	**Unregistered names cannot be renewed.**

Rule Name	Status	Description	Link to rule report
com_p6_renew_available_name_reverts	Verified	*If a name is available (treated as unregistered / free to register), then calling `renew` must revert. You cannot renew a name that the registrar/registry considers still "available" rather than an active registration to extend.*	Initial commit Final commit

COM_P7. Non-zero rent and correct payment

Status: Verified after fix	**The base rent is non-zero and is correctly charged from the user**

Rule Name	Status	Description	Link to rule report

com_p7_base_plus_premium_is_positive	Verified after fix	*When queried a supported payment token, the registrar reports a rent quote that is strictly positive, and that* `balBefore - balAfter = base + premium`*. The rule assumes that* `numer` *and* `denom` *for the supported payment token are non-zero.*	*Initial commit* *Final commit*

Denial of Service

Module General Assumptions

Module Properties

DOS_P1. Role assignment reverts once role count is 15

Status: Verified	**Role assignment reverts once role count is 15**

Rule Name	Status	Description	Link to rule report
eac_p3_grant_64_roles_count_increments_or_reverts_at_cap, eac_p3_revoke_64_roles_count_decrements_or_reverts_at_cap eac_p3_grant_held_and_unheld_roles_count_only_new_increments eac_p3_revoke_held_and_unheld_roles_count_only_held_decrements	Verified	*See EAC_P3*	Initial commit - *Report for grant (unheld only)* - *Report for revoke (held only)* - *Report for grant (held and unheld)* - *Report for revoke (held and unheld)* *Final commit*

DOS_P2. Unregistered names cannot be renewed

Status: Verified	**Unregistered names cannot be renewed.** Assumptions: timestamps fit within `max_uint64`

Rule Name	Status	Description	Link to rule report

dos_p2_commitment_slot_reopens_after_max_commitment_age	Verified	*If a commitment hash already has a non-zero stored commit time `t0 = commitmentAt(commitment)`, and time has moved forward to `t0 + MAX_COMMITMENT_AGE()`, then calling `commit(commitment)` again must successfully refresh the slot in the sense that the stored timestamp is updated: `commitmentAt(commitment) == block.timestamp`.*	*Initial commit* *Final commit*

Enhanced Access Control

Module General Assumptions

The properties are checked on a mock contract that inherits `EnhancedAccessControl.sol`.

Module Properties

EAC_P1. Revoking roles not currently held has no effect

Status: Verified	**Revoking roles not currently held has no effect**

Rule Name	Status	Description	Link to rule report
eac_p1_revoke_nonheld_returns_false	Verified	*if you call `revokeRoles` to clear some `roleBitmap bits` for account on resource, but account currently holds none of those bits, then `revokeRoles` returns `false`, and roles storage is unchanged: `roles(resource, account)` after equals the roles from before the call.*	*Initial commit* *Final commit*

EAC_P2. Role counts are correctly updated

Status: Verified	**For every role `R` in resource `X`, the nybble count in `_roleCount[X]` equals the cardinality of `{account | _roles[X][account & R != 0}`**

Rule Name	Status	Description	Link to rule

			report
eac_p3_grant_64_roles_count_increments_or_reverts_at_cap, eac_p3_revoke_64_roles_count_decrements_or_reverts_at_cap eac_p3_grant_held_and_unheld_roles_count_only_new_increments eac_p3_revoke_held_and_unheld_roles_count_only_held_decrements	Verified	*This is implied by the rule for EAC_P3.*	*See EAC_P3*

EAC_P3. Role counts are between 0 and 15.

Status: Verified	**No role nybble in `_roleCount[resource]` exceeds `0xF (15)` or undershoots 0** The resource and the grantee account are both non-zero. , or

Rule Name	Status	Description	Link to rule report

eac_p3_grant_64_roles_count_increments_or_reverts_at_cap	Verified	*When granting any number > 0 of new roles to a grantee, either each of those role counts increase by one, or at least one of them was 15, in which case the call reverts. The role bitmap does not contain roles already held by the grantee*	*Initial commit* *Final commit*
eac_p3_revoke_64_roles_count_decrements_or_reverts_at_cap	Verified	*When revoking any number > 0 of existing roles from a revokee, either each of those role counts decrease by one, or at least of them was 0, in which case the call reverts. The role bitmap does not contain roles not currently held by the revokee*	*Initial commit* *Final commit*
eac_p3_grant_held_and_unheld_roles_count_only_new_increments	Verified	*When granting a held and an unheld role to a grantee, only the unheld role count increases, or the unheld role count is 15 in which case the call reverts.*	*Initial commit* *Final commit*
eac_p3_revoke_held_and_unheld_roles_count_only_held_decrements	Verified	*When revoking a held and unheld role from a grantee, only the held role count decreases, or the held role count is 0 in which case the call reverts.*	*Initial commit* *Final commit*

EAC_P4. Grant reverts if not settable

Status: Verified	**`grantRoles` reverts if attempting to set a non-settable role** **Assumptions:** `resource != 0`

Rule Name	Status	Description	Link to rule report
eac_p4_grant_reverts_if_not_settable	Verified	*If the caller tries to `grantRoles` with a `roleBitmap` that is not fully contained in what `_getSettableRoles` allows for the HCA-resolved*	*Initial commit* *Final commit*

	sender and at least one requested bit is outside the settable mask, then grantRoles must revert.	

EAC_P5. Role transfer is complete

Status: Verified	**After `transferRoles(resource, src, dst, false)`, `_roles[resource][src] == 0` and the destination holds all source's prior roles.** **Assumptions:** not a self-transfer (`dst != src`), `resource` is non-zero, `src` account has a role to transfer.

Rule Name	Status	Description	Link to rule report
eac_p5_transfer_roles_complete	Verified	*After calling the harness entrypoint exposedTransferRoles(e, resource, src, dst) (which forwards to internal _transferRoles on MockEnhancedAccessControl), role bits on that resource behave as a complete move from `src` to `dst`.* 1. *Source is cleared: `roles(e, resource, src) == 0`* 2. *Destination accumulates by bitwise OR: `roles(e, resource, dst) == (dstBefore | srcBefore)`* *where `srcBefore` / `dstBefore` are the role bitmasks before the transfer.* *"transfer roles merges into `dst` and wipes `src`" for the exposed internal transfer path.*	*Initial commit* *Final commit*

EAC_P6. `grantRoles` reverts if trying to grant root roles.

Status: Verified	**`grantRoles` reverts if trying to grant root roles**

	Assumptions: `account != 0`, `roleBitmap != 0`

Rule Name	Status	Description	Link to rule report
eac_p6_grant_roles_root_reverts	Verified	*Calling `grantRoles` with `resource == 0` (the `ROOT_RESOURCE` id) must always revert. "no grantRoles on root resource" property: root-scoped grants must go through the dedicated grantRootRoles path (as the comment above the rule states).*	*Initial commit* *Final commit*

EAC_P7. Registered token's settable roles do not include admin roles.

Status: Verified	**Registered token's settable roles do not include admin roles.** **Assumptions:** `resource != ROOT_RESOURCE`, `getStatus(anyId) == REGISTERED`

Rule Name	Status	Description	Link to rule report
eac_p7_registered_token_settable_roles_strip_admin_bits	Verified	*For a `REGISTERED` name's non-root EAC resource (`resource = getResource(anyId)`), the registry-specific view of "what roles are settable" matches the underlying EAC mask with all admin grant bits removed. Post-registration, `_getSettableRoles` must not expose admin-granting capability on registered token resources.*	*Initial commit* *Final commit*

EAC_P8. Granting 0 roles returns `false` and does not modify state

Status: Verified	**`_grantRoles` with `roleBitmap == 0` returns `false` and does not modify state.** **Assumptions:** `resource != 0`, `target != 0`

Rule Name	Status	Description	Link to rule report
eac_p8_zero_role_bitmap_no_state_change	Verified	*Calling `grantRoles` with `roleBitmap == 0` is a no-op: `grantRoles` returns `false`. Roles storage is unchanged: `roles(resource, target)` after equals the roles before the call.*	*Initial commit* *Final commit*

ERC1155Singleton

Module General Assumptions

ERC1155 receivers always accept.

Module Properties

ERC_P1. Balance matches owner	
Status: Verified	**ERC1155 token balance of `account` is 1 if and only if `account` is the owner of the token. Otherwise balance is 0**

Rule Name	Status	Description	Link to rule report
erc_p1_balance_matches_owner	Verified	*For any account and token id, `balanceOf(account, id)` is the singleton/binary balance implied by `ownerOf(id)`:* - *If `ownerOf(id) == account`, then `balanceOf( account, id) == 1`.* - *If `ownerOf(id) != account`, then `balanceOf(account, id) == 0`.* *Balances are never outside {0,1} and are fully determined by whether the account is the current owner.*	*Initial commit* *Final commit*

ERC_P2. Singleton tokens are non-divisible	
Status: Verified	**Singleton tokens are non-divisible**

Rule Name	Status	Description	Link to rule report
erc_p2_safe_transfer_reverts_if_value_gt_one	Verified	*For `safeTransferFrom`, if the transferred value is strictly greater than 1, the call always reverts.*	*Initial commit* *Final commit*

ERC_P3. Current, non-expired ids can only be transferred from their `ownerOf`

Status: Verified	**Unauthorized transfers revert** **Assumptions:** transferred value is strictly positive

Rule Name	Status	Description	Link to rule report
erc_p3_safe_transfer_reverts_if_from_not_owner_and_value_positive	Verified	*If `safeTransferFrom` is called with `value > 0`, but `from` is not `ownerOf(id)`, then the call must revert. A positive transfer cannot move tokens unless `from` is the current owner (for that id).*	*Initial commit* *Final commit*

ERC_P4. Zero-valued transfer does not change ownership

Status: Verified	**Zero-valued transfer does not change ownership**

Rule Name	Status	Description	Link to rule report
erc_p4_zero_value_safe_transfer_preserves_latest_owner	Verified	*A `safeTransferFrom` with `value == 0` is an ownership no-op: `ownerOf(id)` before the transfer of `id`, `safeTransferFrom(e, from, to, id, 0, data)` is equal to `ownerOf(id)` afterwards. Zero-amount transfers must not change the token's owner.*	*Initial commit* *Final commit*

ERC_P5. Transfer requires either `from` to be `msgSender` or `msgSender` be approved by `from`

Status: Verified	**Transfer requires either `from` to be `msgSender` or `msgSender` be approved by `from`**

Rule Name	Status	Description	Link to rule report
erc_p5_safe_transfer_reverts_unless_self_or_operator_for_msg_sender	Verified	*Let `sender = _msgSender()`, the HCA-aware effective caller. If `safeTransferFrom` is attempted in a situation where the sender is neither the `from` account nor an operator for `from` (`from != sender`. `!isApprovedForAll(from, sender)`), then `safeTransferFrom` must revert.*	*Initial commit* *Final commit*

ERC_P6. Owners of expired names cannot transfer the name

Status: Violated	**Owners of expired names cannot transfer the name**

Rule Name	Status	Description	Link to rule report
erc_p6_safe_transfer_of_expired_name_reverts	Verified after fix	*If the name/token is expired , then `safeTransferFrom` must always revert, for any `to`, `value`, and `data`, and any `from` that does not hold the role `ROLE_CAN_TRANSFER_ADMIN` on the `ROOT_RESOURCE`. You cannot successfully safeTransferFrom an expired token id unless you hold this privileged role.*	*Initial commit* *Final commit*

ERC_P7. Single and batch operation equivalence

Status: Verified after fix	**Single and batch transfers and balanceOf calls must behave equally** **Assumptions:** deterministic ERC1155 receiver hooks (ALWAYS magic return values) so batch vs single cannot diverge because of receiver nondeterminism.

Rule Name	Status	Description	Link to rule report
erc_p7_pair_batch_equivalent_to_two_singles_balance_of	Verified	*The rule checks that the revert behavior of two `balanceOf` operations is the same as the equivalent batch operation `batchBalanceOf` when given arrays of length two.*	*Initial commit* *Final commit*
erc_p7_pair_batch_equivalent_to_two_singles_safe_transfer	Verified after fix	*The rule checks that the revert behavior and storage effect of two calls to `safeTransferFrom` are the same as the equivalent batch operation `safeBatchTransferFrom` when the latter is given arrays of length two.*	*Initial commit* *Final commit*
erc_p7_single_equivalent_to_singleton_batch	Verified after fix	*The rule checks that the revert behavior and storage effect of one call of `balanceOf` or `safeTransferFrom` is the same as the respective calls to `batchBalanceOf` and `safeBatchTransferFrom` when the latter are given arrays of length one.*	*Initial commit* *Final commit*

Hidden Contract Accounts

Module General Assumptions

Module Properties

HCA_P1. HCA_FACTORY configuration correctness

Status: Verified	**If `HCA_FACTORY.getAccountOwner(sender) != 0`, then `_msgSender()` returns the same address.**

Rule Name	Status	Description	Link to rule report
hca_p1_exposed_msg_sender_equals_get_account_owner_when_nonzero	Verified	*If the HCA factory is configured (`HCA_FACTORY != 0`) and the modeled factory lookup for the transaction sender is non-zero:* - *`sender = msg.sender`* - *`owner = getAccountOwner[sender]` with `owner != 0`* *then `_msgSender()` must equal that owner.*	Initial commit - *Permissioned Resolver* - *HCAContext* - *HCAContext Upgradeable* *Final commit* - *Permissioned Resolver* - *HCAContext* - *HCAContext Upgradeable*

HCA_P2. HCA_FACTORY not configured correctness

Status: Verified	**If `HCA_FACTORY == address(0)`, `_msgSender() == msg.sender`**

Rule Name	Status	Description	Link to rule report

hca_p2_exposed_msg_sender_equals_msg_sender_when_factory_zero	Verified	*If the HCA system is disabled by having `HCA_FACTORY == address(0)`, then `_msgSender()` must not perform any resolution and must equal the raw transaction sender: with a zero factory address, _msgSender() falls back to msg.sender.*	Initial commit - *Permissioned Resolver* - *HCAContext* - *HCAContext Upgradeable* *Final commit* - *Permissioned Resolver* - *HCAContext* - *HCAContext Upgradeable*

Migration

Module General Assumptions

- ERC1155 receiver hooks summarized as `NONDET` (`onERC1155Received` / `onERC1155BatchReceived`), i.e. migration paths that trigger ERC1155 acceptance checks may nondeterministically revert due to receiver behavior.
- In the `Locked` and `WrapperRegistry` migration specs, `VerifiableFactory.deployProxy(...)` is summarized as nondeterministically reverting or returning an arbitrary address.
- In the `Unlocked` migration spec, ENSv1 `BaseRegistrar.reclaim(...)` is summarized as nondeterministically reverting or succeeding. This is to cover the real-world possibility that reclaim can revert.

Module Properties

MIG_P1. Locked migration reverts if name is not locked

Status: Verified	**migrateWrapped reverts if name is not locked (`CANNOT_UNWRAP` fuse not burned)**

Rule Name	Status	Description	Link to rule report
mig_p1_locked_migrate_reverts_if_name_not_locked	Verified	*If an ENSv1 wrapped name is not locked (the v1 `CANNOT_UNWRAP` fuse is not burned), then the Locked migration path must reject it: calling the Locked controller's ERC1155 receiver entrypoint `onERC1155Received` must revert.*	*Initial commit* *Final commit*

MIG_P2. Locked migration reverts if parent node does not match the resolver

Status: Verified	**`_migrateWrapped` reverts if `ids[i] != namehash(parentNode, keccak256(label))`** **Assumptions:** The array of names to migrate is singleton.

Rule Name	Status	Description	Link to rule report
mig_p2_finish_reverts_on_id_or_data_mismatch	Verified	*For the Locked migration flow, calling `finishERC1155Migration(ids, mds)` must revert if the migrated token id does not match the name implied by the migration data label.* *The ERC1155 token id being migrated must correspond to the namehash of the provided label under the wrapped root; otherwise migration is rejected.*	*Initial commit* *Final commit*

MIG_P3. Migration reverts if owner is zero

Status: Verified	**For the locked migration flow, finishing migration must revert if owner is zero** **Assumptions:** The array of names to migrate is singleton.

Rule Name	Status	Description	Link to rule report
mig_p3_finish_reverts_if_owner_is_zero	Verified	*For the Locked migration flow, `finishERC1155Migration(ids, mds)` must revert if the migration data (`mds`) specifies a zero owner:* *Under the condition `mds[0].owner == address(0)`, the call `finishERC1155Migration` must revert.*	*Initial commit* *Final commit*

MIG_P4. Migration reverts if non-zero token approval

Status: Verified	**Migration reverts if `NameWrapper` token has an approval** **Assumptions:** The array of names to migrate is non-empty and at most length two.

Rule Name	Status	Description	Link to rule report
mig_4_finish_reverts_on_nonzero_token_approval	Verified	*For the Locked migration flow, `finishERC1155Migration(ids, mds)` must revert if the ENSv1 `NameWrapper` token being migrated has a non-zero per-token approval and the `CANNOT_APPROVE` fuse has been burned: If `NameWrapper.getApproved(ids[i]) != address(0)` for some `i`, then `finishERC1155Migration` must revert.* *The rationale is that per-token approvals can persist in v1 (especially with `CANNOT_APPROVE` burned) and must not carry over into v2, so the migration path rejects such tokens.*	*Initial commit* *Final commit*

MIG_P5. Migration requires reserved and inherits v2 expiry

Status: Verified	**Migration reverts if `NameWrapper` token has non-zero single-token approval** **Assumptions:** The array of names to migrate is singleton.

Rule Name	Status	Description	Link to rule report
mig_p5_finish_requires_reserved_and_inherits_v2_expiry	Verified	*When a name has not yet been claimed on v2 but exists on v1, and the migration window is open, the name is marked `RESERVED` in v2 with an `expiry`. The rule states the following:* - *Migration is only successful if the name is `RESERVED` in v2, and*	*Initial commit* *Final commit*

	- *Once the name is migrated, its expiry is the one set in v2.*	

MIG_P6. Fuse translation covers all capability-affecting fuse bits.

Status: Verified	**Tested for fuses `CAN_EXTEND_EXPIRY`, `CANNOT_BURN_FUSES`, `CANNOT_CREATE_SUBDOMAIN`, `CANNOT_SET_RESOLVER`, `CANNOT_TRANSFER`**

Rule Name	Status	Description	Link to rule report
mig_p6_fuse_can_extend_expiry_maps_to_role_renew	Verified	`CAN_EXTEND_EXPIRY` *Checks the v1→v2 migration fuse translation for the "extend expiry" capability:* *Compute* `tokenBitmap = tokenRoleBitmapFromFuses(fuses)` *(the v2 role bitmap granted to the migrated token owner). Let* `mask = FUSE_CAN_EXTEND_EXPIRY` *(the v1* `NameWrapper` *fuse bit for "can extend expiry"). Let* `role = ROLE_RENEW` *(the v2 EAC role bit that allows renewal). Then it asserts an exact iff mapping:* `(tokenBitmap has ROLE_RENEW) <=> (fuses has CAN_EXTEND_EXPIRY set), i.e, ((tokenBitmap & role) != 0) <=> ((fuses & mask) != 0)`	*Initial commit* *Final commit*
mig_p6_fuse_cannot_burn_fuses_controls_admin_expansion	Verified	`CANNOT_BURN_FUSES`. *Same as above.*	*Initial commit* *Final commit*
mig_p6_fuse_cannot_create_subdomain_maps_to_role_registrar	Verified	`CANNOT_CREATE_SUBDOMAIN`. *Same as above.*	*Initial commit* *Final commit*

mig_p6_fuse_cannot_set_resolver_maps_to_role_set_resolver	Verified	`CANNOT_SET_RESOLVER`. *Same as above.*	*Initial commit* *Final commit*
mig_p6_fuse_cannot_transfer_maps_to_role_can_transfer_admin	Verified	`CANNOT_TRANSFER`. *Same as above.*	*Initial commit* *Final commit*

MIG_P7. Locked names cannot be migrated through the unlocked path.

Status: Verified	**`_migrateWrapped` reverts if name is locked - locked names must use LockedMigrationController** **Assumptions:** The array of names to migrate is singleton.

Rule Name	Status	Description	Link to rule report
mig_p7_unlocked_migrate_reverts_if_name_locked	Verified	*On the Unlocked migration path, attempting to finish migration for a locked ENSv1 wrapped name must revert. Under the condition* `isLocked(ids[i])` *(the v1 NameWrapper "lock" fuse is burned), the call* `finishERC1155Migration` *must revert.*	*Initial commit* *Final commit*

MIG_P8. Emancipated unmigrated children cannot be registered on v2.

Status: Verified after fix	**`WrapperRegistry.register()` reverts for emancipated children that haven't migrated yet.**

Rule Name	Status	Description	Link to rule report

mig_p8_register_reverts_for_emancipated_non_migrated_child	Verified after fix	*`WrapperRegistry.register(label, ...)` must revert if the label refers to an emancipated child of the wrapped node that has not yet been migrated. The rule encodes those conditions as spec-level predicates:* - *`isEmancipatedChild(label)`: the v1 `PARENT_CANNOT_CONTROL` fuse is burned for the child namehash and label is not a 2LD .eth name.* - *`!isMigratedChild(label)`: in v1 `NameWrapper`, the owner of that child is non-null and there is a non-zero v2 expiry*	*Initial commit* *Final commit*

MIG_P9. Only v1 `BaseRegistrar` may migrate ERC721 tokens

Status: Verified	**`UnlockedMigrationController.onERC721Received` must revert unless the caller is the ENSv1 `BaseRegistrar`**

Rule Name	Status	Description	Link to rule report
mig_p9_onERC721Received_reverts_unless_caller_is_registrar_v1	Verified	*On the Unlocked migration controller, the ERC721 receiver entrypoint `onERC721Received` must reject calls from any sender other than the ENSv1 `BaseRegistrar` (`_REGISTRAR_V1`).* *The rule forces a caller mismatch (`msg.sender != registrarV1Address`), calls `onERC721Received`, and asserts it reverts.*	*Initial commit* *Final commit*

MIG_P10. Only authorized operators can create reservations

Status: Verified	**`BatchRegistrar.batchRegister` can only be called by a privileged role**

Rule Name	Status	Description	Link to rule report
mig_p10_batch_register_only_owner	Verified	*`BatchRegistrar.batchRegister(...)` is protected by `onlyOwner` (OpenZeppelin `Ownable`): any caller who is not `owner()` must cause the call to revert.* *The rule forces `msg.sender != owner`, calls `batchRegister`, and asserts that the call reverts.*	*Initial commit* *Final commit*

MIG_P11. Reserved longer expiry renews instead of registering

Status: Verified	**`BatchRegistrar` renews instead of re-registering if name is `RESERVED` and new expiry > current expiry** **Assumptions:** singleton batch.

Rule Name	Status	Description	Link to rule report
mig_p11_reserved_longer_expiry_renews_not_register	Verified	*If a label is already `RESERVED` in the v2 ETH registry and a `BatchRegistrar.batchRegister` call provides a strictly larger expiry for that label, then any successful `batchRegister` must result in the registry's expiry being updated to exactly that new value (i.e., it must take the renew path, not a re-register path).* *Under the preconditions, after `batchRegister`, the rule asserts: if the call did not revert then `ethGetExpiry(lid) == expires[0]`.*	*Initial commit* *Final commit*

Payment and Pricing

Module General Assumptions

- Math helpers are summarized with mathematical integers

Module Properties

PAY_P1. `rentPrice` reverts for unsupported tokens

Status: Verified	**`rentPrice(label, owner, duration, paymentToken)` reverts if `_paymentRatios[paymentToken].denom == 0`**

Rule Name	Status	Description	Link to rule report
pay_p1_unsupported_token_reverts	Verified	*If a caller asks `rentPrice(label, owner, duration, paymentToken)` for a payment token that is not supported — modeled as `!isPaymentToken(paymentToken)` (i.e., the oracle has no ratio / denom for it) — then `rentPrice` must revert.*	*Initial commit* *Final commit*

PAY_P2. `rentPrice` reverts for zero base units

Status: Verified	**`rentPrice` reverts if `baseRate(label) * duration == 0`**

Rule Name	Status	Description	Link to rule report
pay_p2_zero_base_units_reverts	Verified	*If the "base units" term for pricing is zero, modeled as `baseRate(label) == 0` or `duration == 0` (so `baseRate(label) * duration == 0`), then calling `rentPrice(label, owner, duration, paymentToken)` must revert.*	*Initial commit* *Final commit*

PAY_P3. Prior owners do not pay premium

Status: Verified	**Premium is zero when `owner == address(0)` or `owner == state.latestOwner` – prior owner exemption**

Rule Name	Status	Description	Link to rule report
pay_p3_null_or_prior_owner_zero_premium	Verified	*When there is no "new owner pays premium" case, the premium component returned by `rentPrice` is zero. Concretely, if either:* - *`owner == address(0)`, or* - *`owner == state.latestOwner` where `state = REGISTRY.getState(LibLabel.id(label))`,* *then for a valid supported-token query, `rentPrice(label, owner, duration, paymentToken)` must return `premium == 0`.*	*Initial commit* *Final commit*

PAY_P4. Premium window is finite

Status: Verified	**`premiumPriceAfter(duration)` returns `0` when `duration >= premiumPeriod`**

Rule Name	Status	Description	Link to rule report
pay_p4_premium_after_window_zero	Verified	*If the queried duration is at least the premium window (`duration >= premiumPeriod()`), then the premium-decay helper must have fully decayed to zero: `premiumPriceAfter(duration) == 0`.*	*Initial commit* *Final commit*

PAY_P5. Discount is monotonic

Status: Verified	`integratedDiscount(t)` is monotonically non-decreasing in `t`

Rule Name	Status	Description	Link to rule report
pay_p5_integrated_discount_monotone	Verified	*The oracle's integrated duration-discount function is monotonically non-decreasing in time: If `t1 <= t2`, then `integratedDiscount(t1) <= integratedDiscount(t2)`.*	*Initial commit* *Final commit*

PAY_P6. Invalid labels have no price

Status: Verified	`baseRate(label)` returns 0 for labels with `bytes(label).length == 0` or `> 255`

Rule Name	Status	Description	Link to rule report
pay_p6_empty_or_long_label_base_rate_zero	Verified	*The base-rate function treats invalid label lengths as having zero base rate:* - *Empty label: `baseRate("") == 0`* - *Too-long label: for any `longLabel` with `longLabel.length > 255`, `baseRate(longLabel) == 0`*	*Initial commit* *Final commit*

PAY_P7. Update rejects zero numerator

Status: Verified	Payment ratio integrity – `numer > 0` and `denom > 0` for all active payment tokens

Rule Name	Status	Description	Link to rule report
pay_p7_update_rejects_zero_numerator	Verified	*`updatePaymentToken(paymentToken, numer, denom)` must revert when the update would introduce an invalid ratio with a positive denominator but zero numerator: `denom > 0` and `numer == 0`*	*Initial commit* *Final commit*

Registry and Ownership

Module General Assumptions

- `Arrays.unsafeMemoryAccess` is implemented using a precise summary
- `ERC1155Singleton._asSingletonArrays` is implemented using a precise summary
- `LibLabel.withVersion` is modeled with division, multiplication, and sum

Module Properties

REG_P1. Each ERC1155 token ID has exactly one owner

Status: Verified	**`balanceOf(account, id) == 1` iff `ownerOf(id) == account`**

Rule Name	Status	Description	Link to rule report
reg_p1_erc1155_singleton_balance_iff_owner	Verified	*For any account and token id, the registry's ERC1155-singleton balance semantics hold:* `balanceOf(account, id) == 1` *iff* `ownerOf(id) == account`. *Two "views" are tied together:* `balanceOf` *is exactly the indicator for ownership (binary {0,1}), and there is no way to have balance 1 without being* `ownerOf`*, or be* `ownerOf` *without balance 1.*	*Initial commit* *Final commit*

REG_P2. Transfer completeness

Status: Verified	**Both token ownership and roles transfer correctly**

Rule Name	Status	Description	Link to rule report
reg_p2_transfer_completeness	Verified	*After a successful* `safeTransferFrom(from, to, tokenId, 1, data)` *on a non-expired canonical token:*	*Initial commit* *Final commit*

		1. Ownership moves: `ownerOf(tokenId) == to`. 2. Roles are transferred completely: If `from != to`, then `roles(tokenId, from)` becomes 0. 3. `roles(tokenId, to)` becomes `toRolesBefore | fromRolesBefore` (bitwise OR of prior to roles and from's roles before the transfer). *The seller loses roles (unless self-transfer), the buyer gains them (merged with any existing roles).*	
reg_p2_transfer_completeness_expired_label	verified	*Same as above, but with expired tokens.*	*Initial commit* *Final commit*

REG_P3. Enhanced access control and token version ids are monotonic

Status: Verified	**`eacVersionId` and `tokenVersionId` are monotonically non-decreasing per name.**

Rule Name	Status	Description	Link to rule report
reg_p3_entry_eac_and_token_version_ids_non_decreasing	Verified	`eacVersionId` and `tokenVersionId` are monotonically non-decreasing per name. They increment on register and unregister.	*Initial commit* *Final commit*
reg_p3_token_version_non_decreasing_on_grant_or_revoke	Verified	`tokenVersionId` increases and `eacVersionId` does not change for `grantRoles` and `revokeRoles` (through a call to `_regenerate`).	*Initial commit* *Final commit*

REG_P4. Roles move only on non-zero transfers

Status: Verified	**zero-value transfer must not strip roles. 1-value transfer must strip roles from src and add them to dst.**

Rule Name	Status	Description	Link to rule report
reg_p4_erc1155_zero_transfer_preserves_roles	Verified	*Verify that `safeTransferFrom(..., value = 0, ...)` does not move roles at all. Snapshot `rolesBefore = roles(anyId, account)`, call `safeTransferFrom(e, from, to, id, 0, data)`, and assert that `roles(anyId, account)` is unchanged.*	*Initial commit* *Final commit*
reg_p4b_erc1155_transfer_moves_roles	Verified	*A non-zerol transfer must move role bitmaps from `from` to `to` (with the OR semantics), excluding `from == to`.*	*Initial commit* *Final commit*

REG_P5. Token version bump burns and mints to the same owner

Status: Verified	**Regeneration of token happens through burn and mint** **Assumptions:** Token ids do not overflow `uint32`.

Rule Name	Status	Description	Link to rule report
reg_p5_token_version_bump_burn_mint_same_owner	Verified	*For all name `N` with `tokenVersionId V`: if `tokenVersionId` transitions from `V` to `V+1`, then* - *`_owners[tokenId_V]` must be `address(0)`* - *`_owners[tokenId_V+1]` must equal previous `_owners[tokenId_V]`* - *this transition must be accompanied by exactly one burn of `tokenId_V` and exactly one mint of `tokenId_V+1` to the same owner*	*Initial commit* *Final commit*

REG_P6. Granting and revoking roles increases `tokenVersionId` by exactly one

Status: Verified	**After a successful `grantRoles` / `revokeRoles`, `tokenVersionId` for that entry increases by exactly one.**

Rule Name	Status	Description	Link to rule report
reg_p6_entry_token_version_increase_on_grant_or_revoke	Verified	*After a successful role-change on a name, either `grantRoles(anyId, roleBitmap, account)` or `revokeRoles(anyId, roleBitmap, account)` the name's `tokenVersionId` must increase by exactly 1.*	*Initial commit* *Final commit*

REG_P7. Register reverts if expiry is in the past

Status: Verified	**`register()` reverts if the passed expiry is in the past or the entry's expiry is in the past.**

Rule Name	Status	Description	Link to rule report
reg_p7_register_reverts_when_label_expired_and_arg_expired	Verified	*`register` must revert when both:* - *the label is already expired in the registry, and* - *the expiry argument passed to register is itself already expired.* *Unless `owner == 0`.*	*Initial commit* *Final commit*

REG_P8. Name is available if and only if expired

Status: Verified	**`getStatus(anyId)` returns `AVAILABLE` if and only if `block.timestamp >= entry.expiry`**

Rule Name	Status	Description	Link to rule report
reg_p8_get_status_available_iff_expired	Verified	*For any label id anyId, the registry's status computation matches its expiry predicate exactly: `getStatus(anyId) == AVAILABLE` iff `anyId`'s entry `entry.exipry <= block.timestamp`.*	*Initial commit* *Final commit*

REG_P9. Token operations revert when entry is expired

Status: Verified after fix	**`setSubregistry`, `setResolver`, `grantRoles`, `revokeRoles` on a token resource all revert if `block.timestamp >= entry.expiry`**

Rule Name	Status	Description	Link to rule report
reg_p9_token_role_ops_revert_when_entry_expired	Verified after fix	*For an expired registry entry `anyId`, the token-resource role mutators must be disabled: calling either `grantRoles(resource, roleBitmap, account)` or `revokeRoles(resource, roleBitmap, account)` on that entry's computed token resource `resource = getResource(anyId)` must revert, unless `roleBitmap == 0`.*	*Initial commit* *Final commit*
reg_p9_token_ops_revert_when_entry_expired	Verified	*If a registry entry `anyId` is expired , then the token-resource mutators `setSubregistry(anyId, subregistry)` and `setResolver(anyId, resolver)` must revert, regardless of what subregistry / resolver value is supplied.*	*Initial commit* *Final commit*

REG_P10. Names become immediately available after being unregistrered

Status: Verified	**After `unregister(anyId)`, `entry.expiry == block.timestamp` - name is immediately available**

Rule Name	Status	Description	Link to rule report
reg_p10_unregister_sets_expiry_to_block_timestamp	Verified	*After calling `unregister(anyId)`, the name becomes immediately available by setting its stored expiry to `block.timestamp`..*	*Initial commit* *Final commit*

REG_P11. At successful register, status and owner are updated

Status: Verified	**After `register(label, owner, ..., expiry)` with `owner != address(0)`, `ownerOf(tokenId) == owner` and `getStatus(labelId) == REGISTERED`.**

Rule Name	Status	Description	Link to rule report
reg_p11_register_nonzero_owner_sets_owner_and_registered_status	Verified	*After a successful `register(label, owner, registry, resolver, roleBitmap, expiry)` with a non-zero owner, the name's status becomes `REGISTERED`, and the name's canonical token id is owned by that owner: `ownerOf(tokenId) == owner`.*	*Initial commit* *Final commit*

REG_P12. `register` with null-owner reserves

Status: Verified	**After `register(label, address(0), ..., expiry)`, `ownerOf(tokenId) == address(0)` and `getStatus(labelId) == RESERVED`**

Rule Name	Status	Description	Link to rule report
reg_p12_register_zero_owner_sets_no_owner_and_reserved_status **reg_p12_register_zero_owner_past_expiry_sets_available_status**	Verified	*After calling* `register(label, address(0), registry, resolver, roleBitmap, expiry)` *(the reserve form of registration), the name's status becomes* `RESERVED`*, unless past expiry, in which case it becomes* `AVAILABLE`*. The canonical token id for the label has no owner:* `ownerOf(tokenId) == address(0)` *.*	*Initial commit* *Final commit*

Resolution

Module General Assumptions

- HCA factory call `getAccountOwner` is dispatched to `MockHCAFactoryBasic`.

Module Properties

RES_P1. Subregistry is zero for expired labels.

Status: Verified	**`getSubregistry(label)` returns `address(0)` for expired names**

Rule Name	Status	Description	Link to rule report
res_p1_get_subregistry_zero_when_expired	Verified	*If a label is expired according to the registry, then its resolution pointer to a child registry is cleared: `getSubregistry(label) == address(0)`.*	*Initial commit* *Final commit*

RES_P2. Resolver is zero for expired labels.

Status: Verified	**`getResolver(label)` returns `address(0)` for expired names**

Rule Name	Status	Description	Link to rule report
res_p2_get_resolver_zero_when_expired	Verified	*If a label is expired in the registry, then its resolver pointer is cleared: `getResolver(label) == address(0)`.*	*Initial commit* *Final commit*

RES_P3. Resolver resorts to v1 names for migratable children

Status: Verified	**`WrapperRegistry.getResolver(label)` returns `V1_RESOLVER` for migratable children - ENSv1 resolution fallback during migration**

Rule Name	Status	Description	Link to rule report
res_p3_get_resolver_returns_v1_when_migratable_child	Verified	*During migration, if a label is classified as a migratable child of the wrapped node, then `WrapperRegistry.getResolver(label)` must return the immutable ENSv1 fallback resolver: `getResolver(, label) == V1_RESOLVER()`.*	*Initial commit* *Final commit*

RES_P4. `setParent` cannot be called without privileges

Status: Verified	**`setParent` requires `ROLE_SET_PARENT` on `ROOT_RESOURCE` - canonicity back-pointer is privilege**

Rule Name	Status	Description	Link to rule report
res_p4_set_parent_reverts_without_root_set_parent_role	Verified	*`setParent(parent, label)` must revert unless the HCA-aware sender holds the root-scoped permission to do it: If `msgSender()` does not have `ROLE_SET_PARENT` on `ROOT_RESOURCE`, then `setParent(parent, label)` must revert.*	*Initial commit* *Final commit*

Resolver

Module General Assumptions

- HCA sender resolution is modeled via a ghost summary
- ERC165/interface checks are intentionally under-modeled, allowing both failures and successes.
- `coinType` is either 0 or 60

Module Properties

RSLV_P1. Clearing records resets profile getters

Status: Verified	**After `clearRecords(node)`, all prior records for that node are inaccessible via the public getters.**

Rule Name	Status	Description	Link to rule report
rslv_p1_clear_records_resets_profile_getters	Verified	*After a successful `clearRecords(node)`, the resolver's active record slot moves to a fresh mapping entry (`_records[node][_versions[node]]`), so all getters read empty or zero for that node.. Note that alias data or unrelated getters are not reset.*	*Initial commit* *Final commit*

RSLV_P2. `onlyPartRoles` is a 4-step fallback over EAC resources.

Status: Verified	**The onlyPartRoles modifier checks 4 resources in fallback order**

Rule Name	Status	Description	Link to rule report
rslv_p2_only_part_roles_fallback_resources_grant	Verified	*Formalizes the semantics of the resolver's `onlyPartRoles(node, part, roleBitmap)` permission check as a 4-step fallback over EAC resources, and proves the "grant vs deny" conditions are exact complements.*	*Initial commit* *Final commit*

	Permission is `Granted` iff the user holds `roleBitmap` on any of: - *`resource(node, part)`* - *`resource(0, part)`* - *`resource(node, 0)`* - *`ROOT_RESOURCE`* *Permissioned is `Denied` iff the user holds roleBitmap on none of those four.* *The assertion is `denied <=> !granted`,* *i.e. the fallback sequence is the complete access condition: access is granted exactly when at least one of the four resources grants it*	

RSLV_P3. No record setter bypasses the permission check

Status: Verified	**Every record setter (`setAddr`, `setText`, `setContenthash`, `setABI`, `setPubkey`, `setName`, `setInterface`) enforces the `onlyPartRoles` modifier**

Rule Name	Status	Description	Link to rule report
rslv_p3_record_setters_enforce_onlyPartRoles	Verified	*For each resolver record setter (`setAddr`, `setText`, `setContenthash`, `setABI`, `setPubkey`, `setName`, `setInterface`), the rule checks that the setter cannot succeed unless the HCA-aware sender passes the `onlyPartRoles` permission check for the appropriate `(node, part)`.*	*Initial commit* *Final commit*

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.